FRISMN

Project Requirements Document

CLEAN-SLATE EDITION | V1.0

FRISMN — The Market Learning Ecosystem

A standalone brief for a developer starting with no repository, no code, and no prior project handover.

Purpose: define what to build, for whom, how it should behave, and how completion will be accepted.

Document Control

Field	Value
Product	FRISMN — The Market Learning Ecosystem
Document	Clean-Slate Project Requirements Document
Version	1.0
Audience	Product owner, software developer, technical lead, UX, QA, AI/ML engineer
Starting assumption	Developer has received no code, repository, dashboard, backend, or previous technical package.
Design assets	Product owner will provide/identify the approved final design and source assets separately. Do not invent a replacement design.
Status	Baseline requirements; open decisions are explicitly listed near the end.

1. Product Vision

FRISMN is a Market Learning Ecosystem that helps people understand how they learn about markets, identify relevant learning needs through a structured process, discuss insights with a human Guide, follow a personalised development path, practise, collect evidence, and review progress over time.

FRISMN is not a stock-tip service, signal service, trading terminal, automated investment adviser, or merely a video/course platform. The product focuses on learning, reflection, deliberate practice, evidence, human guidance, and adaptation.

2. Product Outcomes

- Provide a clear public-facing explanation of FRISMN and a reliable entry point into screening/Discovery.
- Run a structured, governed Discovery experience and retain its evidence safely.
- Support human-led review through a Guide workspace.
- Provide each learner with an approved personalised development plan (PDP), learning tasks, practice, assessment, and progress history.
- Give administrators secure controls for staff/accounts, operational visibility, and audit review.
- Use AI to assist observation, decision preparation, and content generation while preserving human authority.
- Create a controlled path for LLM integration and future FRISMN-specific model evaluation/fine-tuning without claiming training that has not happened.

3. Core Product Principles

- Human agency: the learner and human Guide remain central to consequential learning decisions.
- AI proposes; authorised humans approve where approval is required.
- No silent progression: approved learner levels, competencies, and PDPs cannot change without the defined approval workflow.
- Evidence first: AI must distinguish evidence, interpretation, uncertainty, and missing information.
- No manufactured problems: a valid result may be that no significant learning concern is found.
- Deterministic governance: hard rules, permissions, sufficiency checks, and approval gates are enforced by application logic—not left to an LLM prompt alone.
- Transparency: distinguish deterministic output, simulated/mock AI, live LLM output, and a genuinely trained model.
- Privacy by design: minimise personal data, enforce consent and access controls, and do not use raw learner conversations for training by default.
- Auditability: consequential decisions, approvals, revisions, and AI runs must be traceable.

4. User Roles

Role	Purpose	Main permissions
Public visitor	Understand FRISMN and begin screening or request a conversation.	View public pages; submit permitted public forms.
Learner	Complete assigned learning activities and review own development.	Access own profile, tasks, reflections, evidence, approved PDP and progress.

Guide	Review learners, interpret evidence with them, and approve learning decisions.	Access assigned/authorised learners; review evidence; approve/edit/request changes/reject drafts.
Administrator	Operate the system and manage authorised accounts/settings.	Role-controlled account provisioning, operational metrics, configuration and audit access.
Curriculum owner (may be an authorised Admin role initially)	Govern permanent competency library and rubric versions.	Approve core competency/rubric changes and their effective versions.

5. End-to-End Product Journey

1. Public visitor arrives at the FRISMN website and understands the product purpose and boundaries.
2. Visitor starts the approved screening/Discovery entry flow or requests a conversation.
3. Discovery runs using the canonical question bank and deterministic domain rules.
4. The system evaluates completeness/sufficiency and presents a permitted outcome; it does not force an artificial diagnosis or recommendation.
5. Initial evidence and AI-generated observations are prepared for authorised human review.
6. A human Guide reviews the brief and conducts the FRISMN conversation.
7. Learner and Guide identify and agree relevant learning needs through the human-led process.
8. The system drafts a personalised development plan and associated learning routine.
9. The Guide reviews and explicitly approves, edits and approves, requests changes, or rejects the draft.
10. Only the approved PDP becomes active; the learner follows tasks and practice activities.
11. Reflections, practice artefacts, and assessments become evidence for review.
12. The system proposes continuation, reinforcement, clarification, reassessment, progression, PDP revision, or Guide review as appropriate.
13. Every consequential revision follows the approval and versioning rules; the journey repeats over time.

6. Required Application Surfaces

6.1 Public website

- Public pages explaining the FRISMN concept, how the process works, what a Guide does, and the appropriate call to action.
- Approved design and copy will be supplied by the product owner; the developer must implement it faithfully rather than redesigning it.
- All tabs, links, menu items, buttons, and CTAs must work or display a clearly defined disabled/unavailable state.
- Booking/contact forms must connect to a real backend endpoint and report success only after confirmed server success.

6.2 Discovery experience

- Implement the canonical 34-question bank with approved wording and sequence rules.
- Store Discovery sessions and responses with secure session identity and timestamps.
- Support progress/resume behaviour only where specified and safe.
- Apply deterministic contradiction, completeness, sufficiency, and exit-pathway rules.
- AI must not autonomously rewrite, skip, or choose Discovery questions unless the product owner approves a future change.
- Show neutral, understandable prompts; do not label a learner with unsupported diagnoses or create a problem where evidence does not support one.

6.3 Learner dashboard

- Secure sign-in and access to the learner's own information only.
- Show current approved PDP, plan version, tasks, practice activities, reflections, submitted evidence, assessment outcomes, and progress timeline.
- Allow the learner to submit reflections and practice evidence through persistent backend APIs.
- Clearly distinguish pending drafts from approved plans and demo content from live content.
- Do not expose internal AI prompts, other learners' information, Guide-only notes, or Admin tools.

6.4 Guide dashboard

- Secure Guide sign-in and server-side role-based authorisation.

- Roster/search/filter for authorised learners, review queue, evidence timeline, and learner brief.
- Display AI observations and recommendations as proposals with supporting evidence and uncertainty.
- Support approve, edit-and-approve, request changes, and reject with a reason where appropriate.
- Keep pending reviews visible; while a revision is pending, the previous approved PDP remains active.
- Record the Guide, timestamp, decision, version, and reason for each consequential action.

6.5 Admin dashboard

- Secure, role-protected administrative workspace.
- Authorised account/staff provisioning, role management, operational metrics, configuration, and audit access.
- Enforce permissions on every backend API; hiding a button is not sufficient.
- Record privileged actions in the audit trail.

7. Three-Agent AI System

Implement three specialised agents coordinated by an orchestrator. The application must define explicit contracts, inputs, outputs, allowed actions, timeouts, retries, validation, and trace IDs. Agents may be separate modules/services; they do not need to be three independent foundation models. The design should permit model/provider selection without allowing an agent to bypass deterministic governance.

Agent / component	Primary responsibility	Required output	Forbidden authority
Orchestrator	Own workflow state, agent order, context packaging, retries, timeouts and traceability.	Run status, stage results, errors, trace/correlation ID.	Cannot bypass policy, permissions, or human approval.
1. Observational AI	Summarise supplied Discovery/evidence; identify supported patterns, gaps, contradictions and uncertainty.	Structured observations with evidence/source references and uncertainty.	Cannot invent facts, diagnose, or make final learner-level decisions.
2. Decision AI	Propose a permitted next learning/review action against approved criteria and deterministic results.	Structured proposal, rationale, evidence links, review state.	Cannot publish a PDP, change an approved level, or override sufficiency/approval rules.
3. Generative AI	Draft Guide briefs, explanations, candidate learning activities and candidate PDP text.	Clearly labelled draft linked to approved evidence/context.	Cannot present draft as approved, or generate stock tips/trade signals.
Validator / safety layer	Validate schema, grounding, prohibited content, permissions, required fields and consistency.	Pass/fail, issues, and audit result.	Cannot silently make an unlogged consequential decision.

8. Required AI Workflow

14. Create an authorised, minimised, versioned evidence package from the Discovery session.
15. Run deterministic rules for question flow, completeness, sufficiency, and permitted exits.
16. Call Observational AI with only the evidence/context it is authorised to access.
17. Validate its structured output and evidence references.
18. Call Decision AI to propose an action within the deterministic rule boundaries.
19. Validate the proposed action against application rules and current approval state.
20. Call Generative AI to draft the required brief or learning artefact from validated inputs.
21. Validate the draft for schema, evidence support, uncertainty, prohibited advice, and privacy.
22. Persist the run, agent/model versions, source references, validation result, and trace ID.
23. Present consequential proposals to the Guide for review; do not auto-approve them.
24. If a provider fails, times out, or returns invalid output, show a safe error or explicitly labelled mock result. Never fabricate a live success.
25. After human review and conversation, prepare a proposed PDP; activate only after explicit approval.
26. Use new practice/assessment evidence to propose review/adaptation under the same governance.

9. LLM Integration Requirements

- Create a provider abstraction so the application is not tightly coupled to one vendor.

- Support a mock/development mode and a separately configured live-provider mode.
- Clearly label mock/simulated output in the UI and logs.
- Keep provider credentials on the server using environment/secret management; never expose them in frontend bundles.
- Use structured output schemas, bounded input/output sizes, timeouts, limited retries, error handling, and usage/cost logging.
- Ground prompts in approved, versioned FRISMN knowledge and authorised learner evidence.
- Treat user content and retrieved documents as untrusted data; they cannot override system instructions, permissions, or approval rules.
- Log provider/model version, prompt/template version, trace ID, validation status, and evidence references subject to privacy rules.
- Allow the product owner to configure provider/model and cost limits after technical and privacy review.

10. FRISMN Learning and Model-Training Pipeline

LLM API integration is not the same as training a model. The system must first support governed collection and evaluation of high-quality examples. Proprietary fine-tuning is a separate future release gate and must not be represented as complete unless a model has actually been trained, evaluated, and approved.

- Collect training candidates only where there is an appropriate legal basis and consent.
- Use Guide-verified and curated examples; raw learner chats and unreviewed AI outputs must not automatically enter training datasets.
- De-identify and minimise personal information; define retention, deletion, and access policies.
- Store provenance, consent state, curator, rubric version, dataset version, inclusion/exclusion decision, and reason.
- Provide a controlled export process for approved examples (for example, JSONL) with an audit record.
- Maintain separate train/validation/test splits and prevent data leakage.
- Create benchmark cases for evidence grounding, uncertainty, contradictions, privacy, fairness, prohibited financial advice, and consistency.
- Version model/provider configuration, prompts, datasets, evaluation results, and release decisions.
- Guide feedback goes into a review queue; no unchecked automatic model-weight updates.
- Do not claim a proprietary FRISMN-trained model exists until training, benchmark evaluation, and model release approval are evidenced.

11. PDP, Competency and Evidence Governance

- Maintain separate entities for Student Learning Profile, current approved level, and PDP.
- Suggested levels: L1 Foundational Understanding; L2 Guided Application; L3 Independent Application; L4 Adaptive Mastery. Detailed rubric criteria must be approved before real progression decisions use them.
- Use FRISMN core competencies and separately tracked learner-specific competency proposals.
- Approving a learner-specific competency must not automatically modify the permanent core competency library.
- Permanent core library/rubric changes require authorised curriculum-owner/Admin approval, versioning, and audit history.
- Evidence status values: Sufficient, Insufficient, Conflicting, Review required.
- Evidence sufficiency must use approved competency criteria plus analysis of multiple evidence sources and uncertainty.
- Do not invent arbitrary score or attempt thresholds. Formal assessments and casual reflections must not have equal weight unless the approved rubric explicitly says so.
- AI may propose Continue, Reinforce, Clarify, Reassess, Propose progression, Propose revision, or Pause for Guide.
- Every PDP revision needs fresh explicit Guide approval before activation. Keep the prior approved version active while a revision is pending.

12. Minimum Data Entities

- Users, roles, account status, sessions/tokens, consent records.
- Discovery question-bank versions, Discovery sessions, responses, sufficiency outcomes.
- Evidence items, source references, reflections, tasks, practice submissions, assessment results.

- Student Learning Profile, competency library, learner-specific competency proposals, rubric versions, competency decisions.
- PDP versions, lifecycle states, Guide reviews, approvals, rejections and reasons.
- AI runs, agent stages, provider/model and prompt versions, validation results, traces.
- Training candidates, curation decisions, dataset versions, exports, benchmark runs, release decisions.
- Audit events, operational logs, and security events.

13. Security, Privacy and Reliability

- Enforce authentication and role-based access control on every protected backend endpoint.
- Use secure password hashing, suitable token/session lifetime, safe storage, and CSRF protection where required by the selected auth approach.
- Validate all inputs server-side and rate-limit public forms and AI endpoints.
- Use least-privilege database accounts; do not use a database superuser for application runtime.
- Do not log secrets or unnecessary personal data.
- Provide structured logs, correlation IDs, health checks, and error monitoring appropriate to the chosen deployment.
- Document backup/restore, migration and rollback procedures, data retention/deletion, and incident response.
- Fail safely: an AI/provider failure must not change learner state or create a fake approved output.

14. Technical Implementation Expectations

The developer may propose an appropriate implementation stack, but must submit the architecture and deployment proposal for product-owner approval before locking in major infrastructure decisions. Prefer a maintainable web application with a separate frontend, backend API, persistent database, and modular AI layer. Keep business rules separate from provider-specific AI code.

- Provide reproducible local development setup, environment template without secrets, database migrations, seed/demo data, and start scripts.
- Use version control, code review, formatting/linting, automated tests, and dependency lockfiles.
- Define API contracts and error formats; avoid frontend dependence on undocumented internals.
- Separate development, test, and production configuration and data.
- Document the production hosting approach, domain/TLS, backups, monitoring, rate limiting, email/booking integrations, and recovery plan.
- Do not introduce paid services or vendor lock-in without documenting cost, purpose, alternatives, and approval.

15. Testing and Acceptance Criteria

Area	Acceptance test required
Public website	All approved pages, tabs, links, forms and CTAs work; failures are visible and honest.
Navigation	Direct route, refresh, back/forward, permissions, not-found and unavailable states tested; no blank pages.
Discovery	Canonical questions, deterministic order, persistence, sufficiency/exit and edge cases tested.
Learner workspace	Learner sees own records only; reflections/tasks persist; approved vs pending PDP is clear.
Guide workspace	Roster/review flow works; approve/edit/request changes/reject is audited; RBAC enforced.
Admin workspace	Privileged APIs blocked for non-admin roles; account and configuration actions audited.
PDP governance	Pending revision leaves previous approved PDP active; no progression without explicit approval.
AI agent contracts	Each agent's schema, allowed actions, grounding and failure cases tested.
Orchestration	Hand-offs, timeouts, malformed output, retries, provider failure and safe fallback tested.
Live LLM	Tested with a configured provider; mock output cannot be mistaken for live output; secrets remain server-side.
Training pipeline	Consent, curation, de-identification, exclusion, dataset versioning, export and audit tested.
Security	Authentication/RBAC, validation, rate limits, secrets handling and privacy controls reviewed.
Release	Clean install, build, tests, database migrations, backup/restore and

	deployment smoke test evidenced.
Handover	Architecture, API, environment, setup, deployment, tests, known limitations and runbook delivered.

16. Delivery Plan

27. Stage 1 — Discovery and specification: confirm open decisions, design inputs, user journeys, data/privacy constraints, and delivery milestones.
28. Stage 2 — UX implementation: build the approved public website and dashboard surfaces; wire navigation and screen states.
29. Stage 3 — Core backend: authentication, roles, Discovery, persistence, booking/contact, data model and audit events.
30. Stage 4 — Human governance: Guide review, competency/evidence records, PDP versioning, approval and rejection flows.
31. Stage 5 — AI foundation: orchestrator, three agent contracts, validator, mock provider, traceability, safe fallbacks and automated tests.
32. Stage 6 — Live LLM: configure the selected provider, test outputs and cost controls, run the evaluation suite.
33. Stage 7 — Training-data pipeline: consented curation, versioning, export, benchmark/evaluation and future fine-tuning gates.
34. Stage 8 — QA and release: end-to-end testing, security review, deployment verification, documentation, owner acceptance.

17. Required Deliverables from Developer

- Approved architecture and implementation plan with milestones and dependencies.
- Working source repository and reproducible setup instructions.
- Public website, Discovery flow, Learner dashboard, Guide dashboard and Admin dashboard.
- Backend APIs, persistent database, migrations, authentication/RBAC, audit trail.
- Three-agent AI runtime, orchestrator, validation, mock/live provider separation and tests.
- LLM integration configuration and governed training-data curation/export pipeline.
- Automated unit, integration, API, role-security and browser end-to-end test suites.
- Deployment/environment guide, API reference, data dictionary, architecture diagram and operational runbook.
- Release report mapping every requirement to PASS, FAIL, BLOCKED, or NOT IMPLEMENTED with evidence.
- Known limitations, security risks, third-party costs, open decisions, rollback steps, and handover notes.

18. Reporting and Definition of Done

- Do not mark work PASS unless it was actually run and evidence can be supplied.
- Separate code implemented, automated tests passed, browser behaviour verified, and production deployment verified.
- Mark mock/simulated features clearly and list APIs that remain stubbed.
- List files/modules delivered, known gaps, risks, and any decisions needed from the product owner.
- Any requirement not completed must be marked BLOCKED or NOT IMPLEMENTED—not implied to be finished.

Definition of Done: the project is accepted only when the agreed user journeys work end-to-end, data persists securely, roles are enforced server-side, Guide approvals govern PDP/level changes, the three-agent pipeline is tested, live LLM mode is accurately distinguished from mock mode, training-data governance is in place, and all release criteria have evidence. A working mock prototype is a milestone, not proof of production readiness.

19. Open Decisions for Kickoff

- Approved design source/assets and exact public-page content to be supplied by the product owner.
- Final technology stack, hosting provider, domain, and production database version.
- Live LLM provider/model, budget ceilings, data-processing region, and provider terms.
- Authentication method, email/SMS/booking integrations, and operational notification channels.
- Detailed competency rubric criteria for L1–L4 and formal assessment design.
- Data retention/deletion schedule and consent wording for learning-data curation.
- Expected launch phases, budget, delivery milestones, and release approvers.
- Fine-tuning dataset quality/volume threshold and model release evaluation criteria.

Appendix A — Starter Requirement IDs

ID	Requirement	Initial status
FRI-PROD-001	Product vision and prohibited scope understood	To confirm at kickoff
FRI-WEB-001	Public website built from approved design assets	Not started
FRI-DISC-001	Canonical Discovery flow and deterministic rules	Not started
FRI-LEARN-001	Learner dashboard and persistent learning activity	Not started
FRI-GUIDE-001	Guide review and approval workflow	Not started
FRI-ADMIN-001	Admin dashboard and RBAC	Not started
FRI-AI-001	Observational Agent	Not started
FRI-AI-002	Decision Agent	Not started
FRI-AI-003	Generative Agent	Not started
FRI-AI-004	Orchestrator, validation, traceability and fallback	Not started
FRI-LLM-001	Live LLM provider integration	Not started
FRI-ML-001	Curated learning-data pipeline	Not started
FRI-ML-002	Fine-tuning and model evaluation release gate	Future phase / not started
FRI-QA-001	End-to-end acceptance and handover evidence	Not started